Relatório Completo: Miniprojeto Jogo da Velha (POO)

Carlos Eduardo e Humberto Henrique – Ciência da Computação (UFMT)

1. Visão Geral

Este projeto implementa o Jogo da Velha com foco estrito em Orientação a Objetos e boas práticas de C++ Moderno. A arquitetura dispensa ponteiros crus, utilizando `std::unique_ptr` para gestão de memória. O sistema contempla divisão modular em `.h`/`.cpp`, três níveis de Inteligência Artificial, persistência de dados para suspensão de partidas e ranking global.

2. Estrutura de Diretórios

A separação de responsabilidades foi feita da seguinte forma:

- `include/`: Contém os cabeçalhos (`.h`), definindo os contratos públicos e permitindo compilação modular.
- `src/`: Contém as implementações (`.cpp`), abrigando a lógica de negócio, IA e o ciclo de execução (`main.cpp`).
- `data/`: Armazena o estado dinâmico gerado, como o `savegame.txt` e `top10.txt`.

O projeto é compilado localmente via `g++`, gerando o executável `jogo_da_velha.exe`, possuindo suporte a `Makefile`.

3. Arquitetura Orientada a Objetos

3.1. Classe Tabuleiro

Encapsula o estado do jogo utilizando `std::array<char, 9>` por sua eficiência sem alocação dinâmica. A classe é inteiramente responsável por validar e aplicar jogadas, converter coordenadas (1-9), verificar condições de vitória (linhas, colunas, diagonais) ou empate, e gerar os dados para serialização.

3.2. Hierarquia de Jogadores

Classe Abstrata `Jogador`: Define o contrato obrigatório com o método virtual puro `escolherJogada(Tabuleiro&)` e um destrutor virtual. Garante o polimorfismo do sistema.
Classe `JogadorHumano`: Implementa a leitura e validação robusta do `std::cin`, permitindo também a entrada '0' para suspender o jogo.
Classe `JogadorComputador`: Base para as IAs, contém gerador de números aleatórios e lógicas de simulação. Dela derivam três comportamentos polimórficos:

- **BotAleatorio (Elementar):** Seleciona casas vazias.
- **BotBasico (Básico):** Tenta vencer imediatamente; caso impossível, joga aleatoriamente.
- **BotMedio (Médio):** Avalia a matriz para vencer, bloquear a vitória iminente do usuário ou jogar aleatoriamente.

3.3. MotorJogo e Smart Pointers

A classe `MotorJogo` orquestra a aplicação. Ela gerencia os turnos e estatísticas armazenando os jogadores em um `std::array<std::unique_ptr<Jogador>, 2>`. A instanciação é feita exclusivamente via `std::make_unique`, eliminando o uso de `new/delete` e evitando vazamentos de memória (*memory leaks*). A inversão de dependência permite que o motor alterne turnos chamando `escolherJogada()` sem conhecer a classe concreta.

4. Funcionalidades e Menu

O menu iterativo, protegido contra entradas textuais inválidas, oferece controle total do fluxo:

1. **Iniciar/Continuar:** Permite resgatar um save ou forçar uma nova partida.
2. **Quem Inicia:** Humano, Computador ou Aleatório.
3. **Dificuldade:** Altera o nível do Bot em runtime.
4. **Suspender:** Invoca o sistema de serialização.
5. **Ranking Top 10:** Exibe histórico de pontuações.

5. Persistência (Nível III)

Savegame: O estado integral (turno, estatísticas, dificuldade e matriz) é gravado em texto. O maior desafio técnico atendido foi a **restauração polimórfica**: ao carregar, o motor lê um identificador de tipo (ex: `bot_medio`) e reinstancia a classe filha correta dinamicamente.
Ranking: O arquivo `top10.txt` salva as estatísticas. A pontuação obedece a regra: Vitória (3 pts), Empate (1 pt) e Derrota (0). A ordenação considera pontos, vitórias e ordem alfabética.

6. Conclusão

O software cumpre integralmente os Níveis I, II e III estabelecidos. O uso de polimorfismo associado à gestão moderna de memória e separação arquitetural estrita resultou em um sistema modular, escalável e seguro contra anomalias de execução, atendendo a todos os critérios de C++ exigidos.